1. SYNOPSIS:

- The Text-Based Python RPG Game is a command-line-based Python program that enables the user to register, login, create a character, navigate different areas, fight enemies, collect items, and earn experience as the game advances

- The game has been created in such a manner that it is very simple and easy to comprehend. The ease of playing the game and understanding the code of the game makes it very enjoyable and fun.

- The code has been developed in modules. The modules are user management module, game module, inventory management module, store module, and utility module. The code is very easy to comprehend.

- The player's information, such as username, password, character information, experience points, money, and inventory, are saved in JSON files. This allows the player to save game progress and resume from where they left off.

- The main purpose of this project is to prove that fun and entertaining games can be created without having to use complicated graphics or development tools. The project proves that a complete game can be created with simple logic and basic concepts using Python.

- The main goal of this project is to make game development process as easy as gaming

- As it is a text-based game, it can be run on any system where Python is installed. There are no special hardware requirements to run this project. The project also proves that basic programming concepts such as functions, loops, decision-making statements, file handling, and modular programming can be used to create a real-time project.

- The goal of this project is to enhance logical reasoning, creativity, confidence, and game development skills using a simple yet effective approach.

2. PREAMBLE

2.1 General Introduction:

- Text-based RPGs (Role-Playing Games) are one of the oldest forms of computer games. These games are based on text prompts and user input. These games are simple yet highly interactive.

2.2 Statement of Problem:

- Programmers who are new to programming often find it difficult to understand how games are created. Most games are either too simple to be used as an example or too complex to be understood. The project aims to solve this problem by creating a simple RPG game using basic Python programming. The game will be simple to understand and learn from.

2.3 Objective and Scope of the study:

- Implement user registration and login functionality.
- Created a system for character level progression and attribute management (health, XP, coins).
- Design and developed various game modules, including hunting, adventure, inventory
- management, a store, and character profiles.
- Incorporate file-based storage for user data persistence.
- Provide a user-friendly text-based interface with clear instructions and feedback.

- **Scope:**
- The project will include core RPG elements such as character development, combat, and
- inventory.
- The game will be text-based, focusing on gameplay mechanics rather than graphical
- presentation.
- User data will be stored in a JSON file.
- The game will be developed using Python 3.x.

2.4 Module Description with functionality:

2.4.1 User Management Module:

- Description: This module handles user registration, login, and deletion.
- Functions:
- register(): Creates a new user account, storing the username and password.
- login(): Authenticates existing users and retrieves their data.
- delete_user(): Removes a user account from the system.
- load_user_data(): Loads user data from the JSON file.
- save_user_data(): Saves user data to the JSON file.

2.4.2 Game Mechanics Module:

- Description: This module contains the core gameplay functions, including hunting,
- adventure, chopping, and reward systems.
- Functions:
- rpg_hunt(): Simulates a hunting expedition where the player encounters creatures, gains
- rewards, and loses health.
- rpg_adventure(): Presents a more challenging adventure scenario with a single, powerful
- creature.
- rpg_chop(): Allows the player to chop wood and gather resources.
- reward(): Provides random rewards to the player.
- get_level(): Calculates the player's level based on their XP.
- check_level_up(): Checks if the player has leveled up and updates their stats

2.4.3 Inventory Module:

- Description: Manages the player's items.
- Function:
- view_inventory(): Displays the items currently in the player's inventory.

2.4.4 Profile Module:

- Description: Displays the player's character information.
- Function:
- view_profile(): Shows the player's username, level, stats (attack, defense, health), coins, and XP.

2.4.5 Store Module:

- Description: Allows the player to buy items from a store.
- Function:
- store(): Displays the store's inventory and handles purchases.

2.4.6 Main Game Module:

- Description: The main entry point of the game, orchestrating the game loop and user
- interaction.
- Function:
- start_game(): Manages the main game loop, user input, and calls other modules.

2.4.7 Utils Module:

- Description: Provides utility functions and data structures used across other modules.
- Functions/Data:
- get_level(): Calculates the player's level.
- item_emojis: A dictionary mapping item names to emojis for display.

3. Review of Literature

3 .1 Review of Literature :- Basic Theory of the Game

- Role-playing games work on the idea that a player starts with limited abilities and improves over time by playing the game. As the player performs actions like fighting enemies or collecting items, they gain experience, level up, and become stronger.
- Most role-playing games (RPGs) use concepts like health, experience points, inventory, and equipment to measure the progress of players. This project follows the same basic idea but retains the text-based design to ensure easy understanding and learning of the game logic.

3.2 Analysis of Existing Similar Systems: Zork

- Zork, which is one of the earliest examples of a similar system, is a popular text-based adventure game that was developed in the late 1970s. In this game, the player has to control the character in a fantasy world by entering commands in text format.
- The game was quite immersive and engaging, despite the lack of graphics. The current study takes cues from the tradition of text-based adventure games like Zork, but with the addition of modern role-playing game elements like combat, character development, crafting, and inventory management.

4. Technical Description

4.1 Hardware Requirements

- Processor: Intel / AMD processor or equivalent
- RAM: Minimum 2 GB (4 GB or more recommended)
- Storage: 200–300 MB free disk space to support future game data, save files, logs, and feature expansions
- Display: Terminal or console window
- Input Device: Keyboard

4.2 Software Requirement

- Windows: Windows 10 or later
- Linux: Modern Linux distributions
- macOS: macOS 11 or later

5. System Design and Development

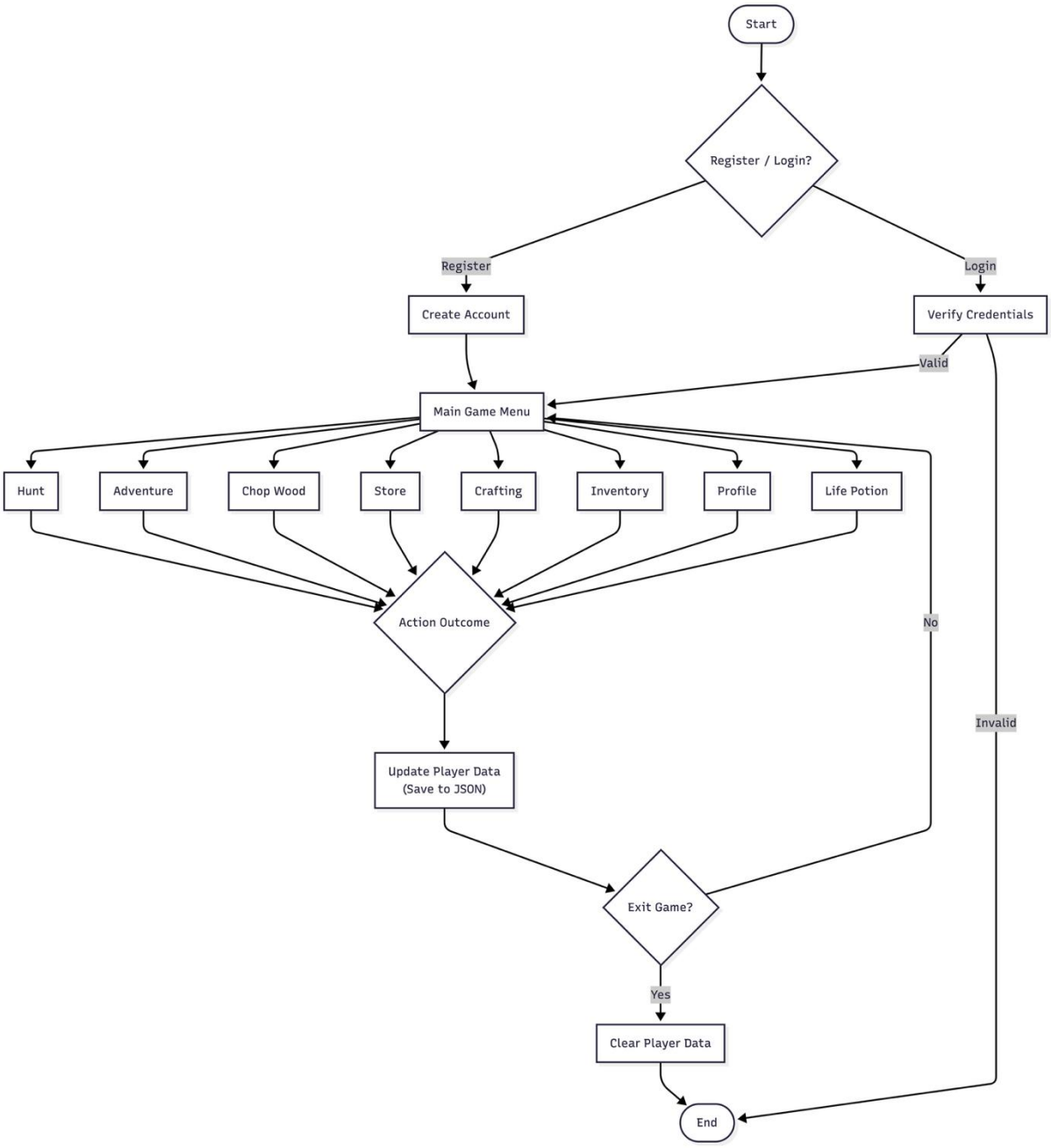

Figure 5.1 Flowchart:

Data flow diagram:

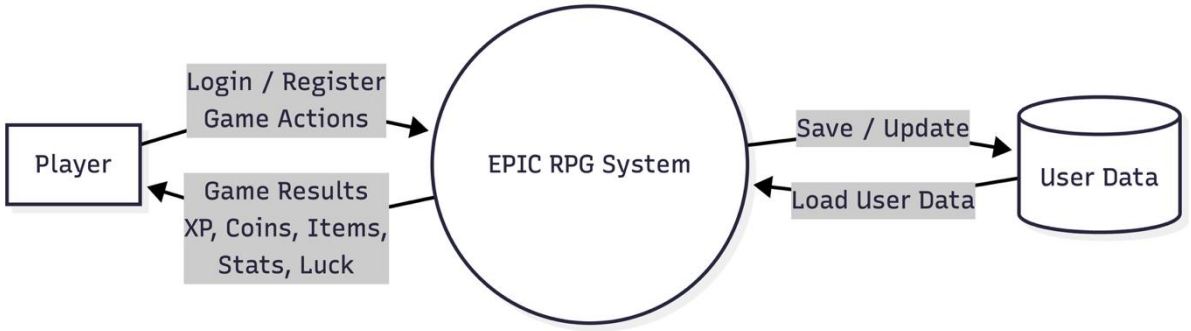

Figure 5.2 DFD-0

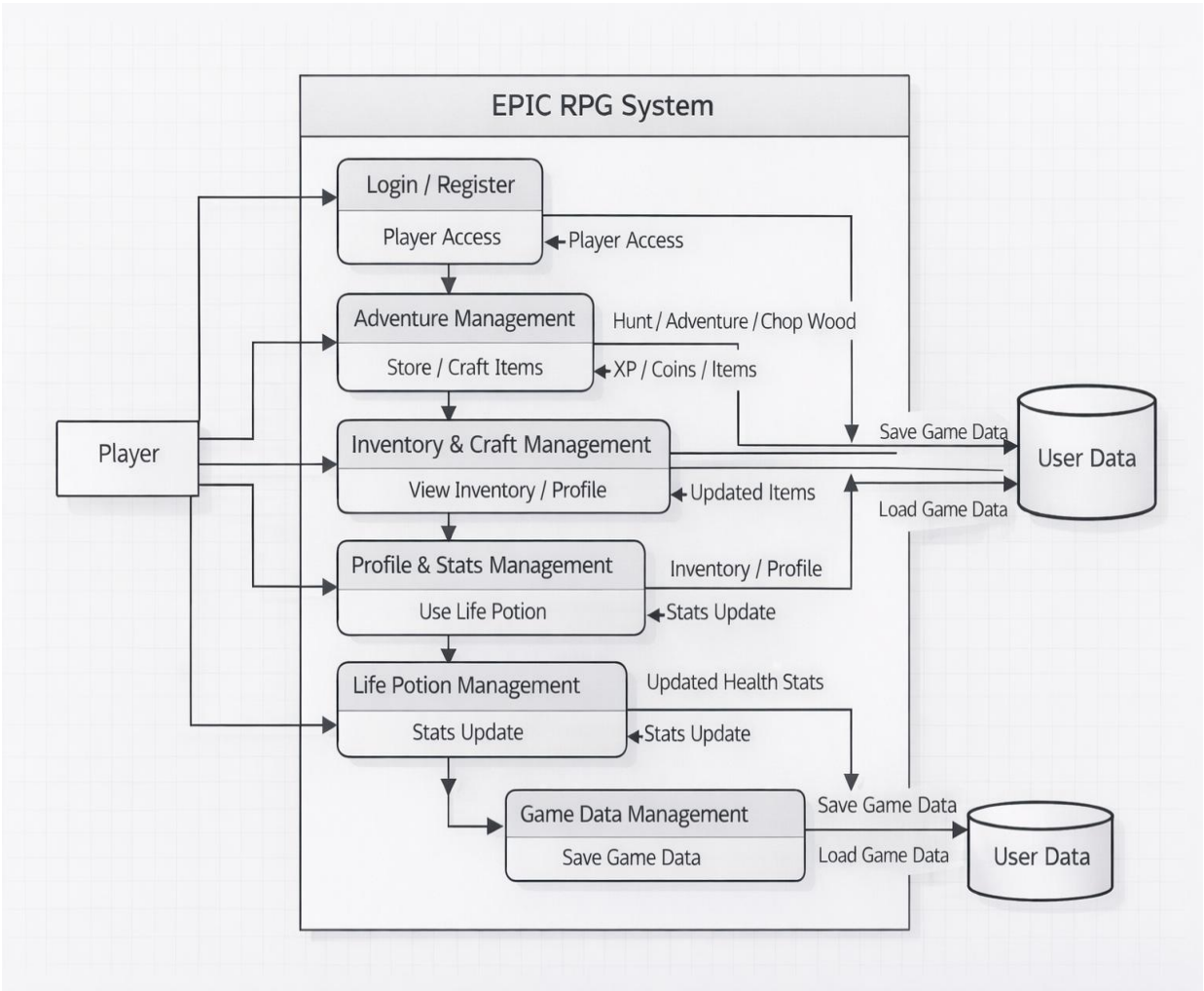

Figure 5.3 DFD Level-1

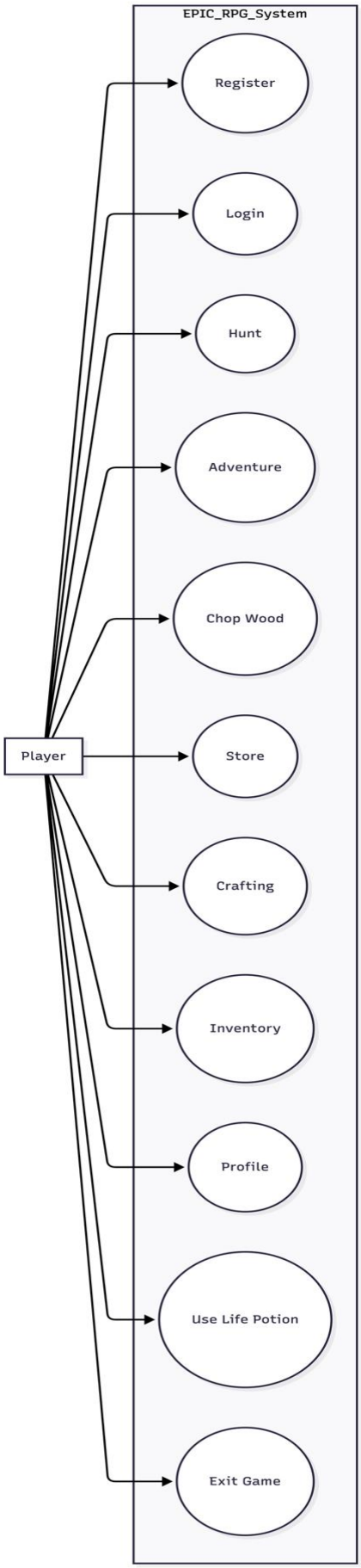

Figure 5.4 Use case diagram

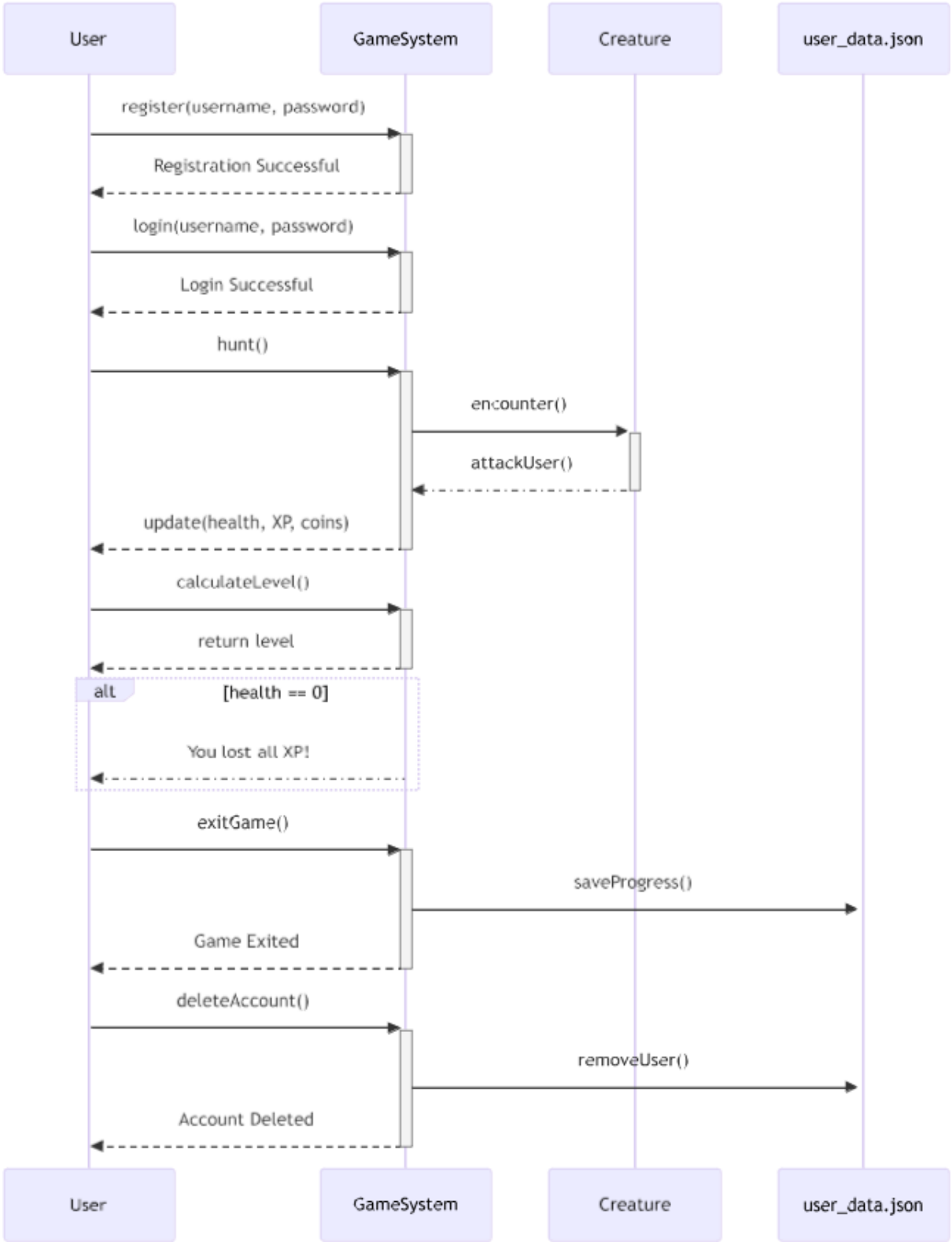

Figure 5.5 Sequential diagram

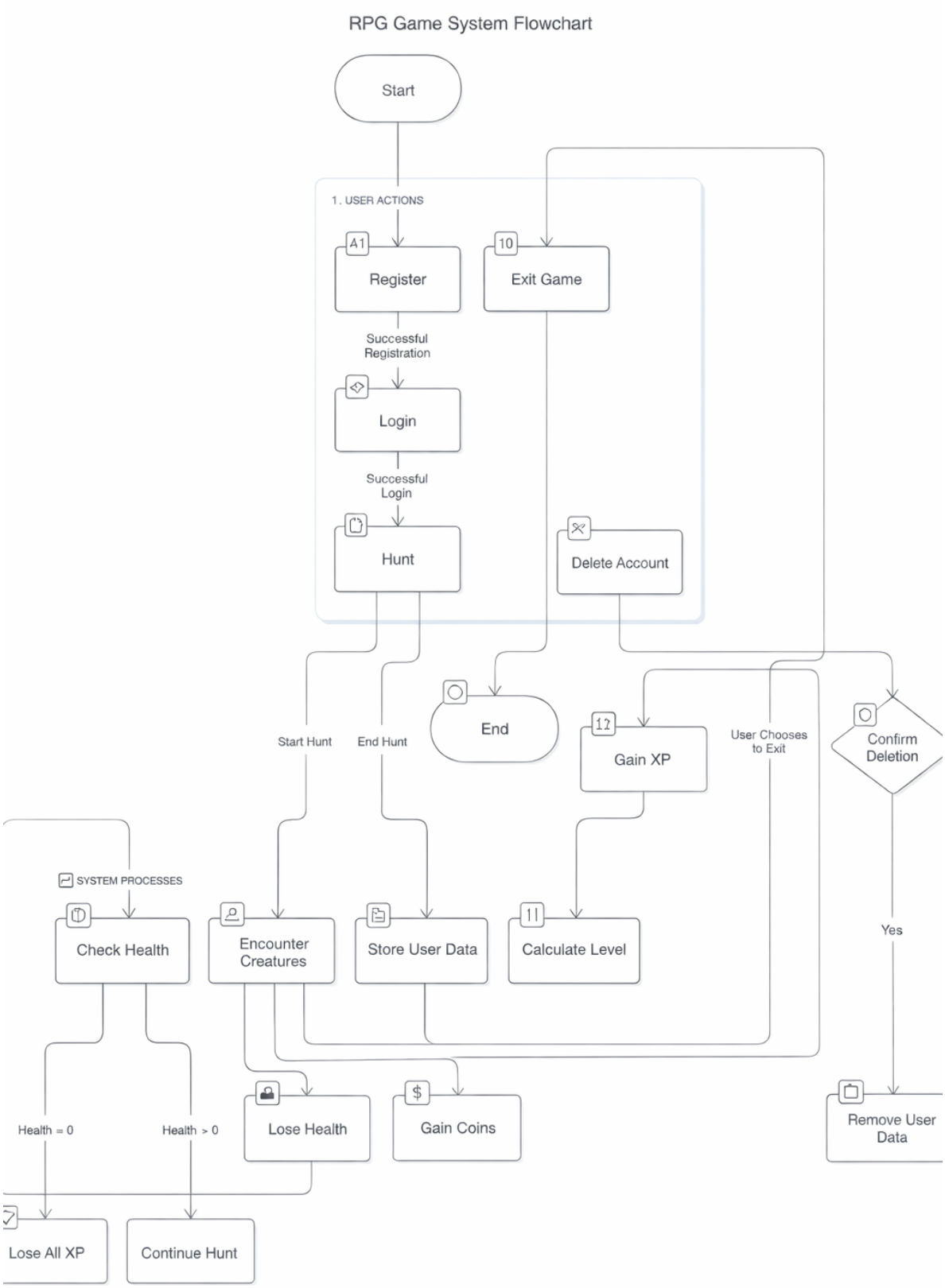

Figure 5.6 Activity diagram:

5.1 Menu Design

- The menu is the main place where the player controls the game. It shows all the available actions in a simple numbered list so the player can easily decide what to do next.
- The player just enters the number of the option they want, and the game performs that action. After the action is completed, the menu is shown again so the player can continue playing without any confusion.

MENU
1. 🎯 Hunt
2. 🗺️ Adventure
3. 🧪 Heal using Life Potion
4. 🎒 View Inventory
5. 👤 View Profile
6. 🏪 Store
7. 🪵 Chop Wood
8. 🎁 Reward
9. 🛠️ Craft
10. 🗑️ Delete User
11. 🚪 Exit

5.2 Screen Design

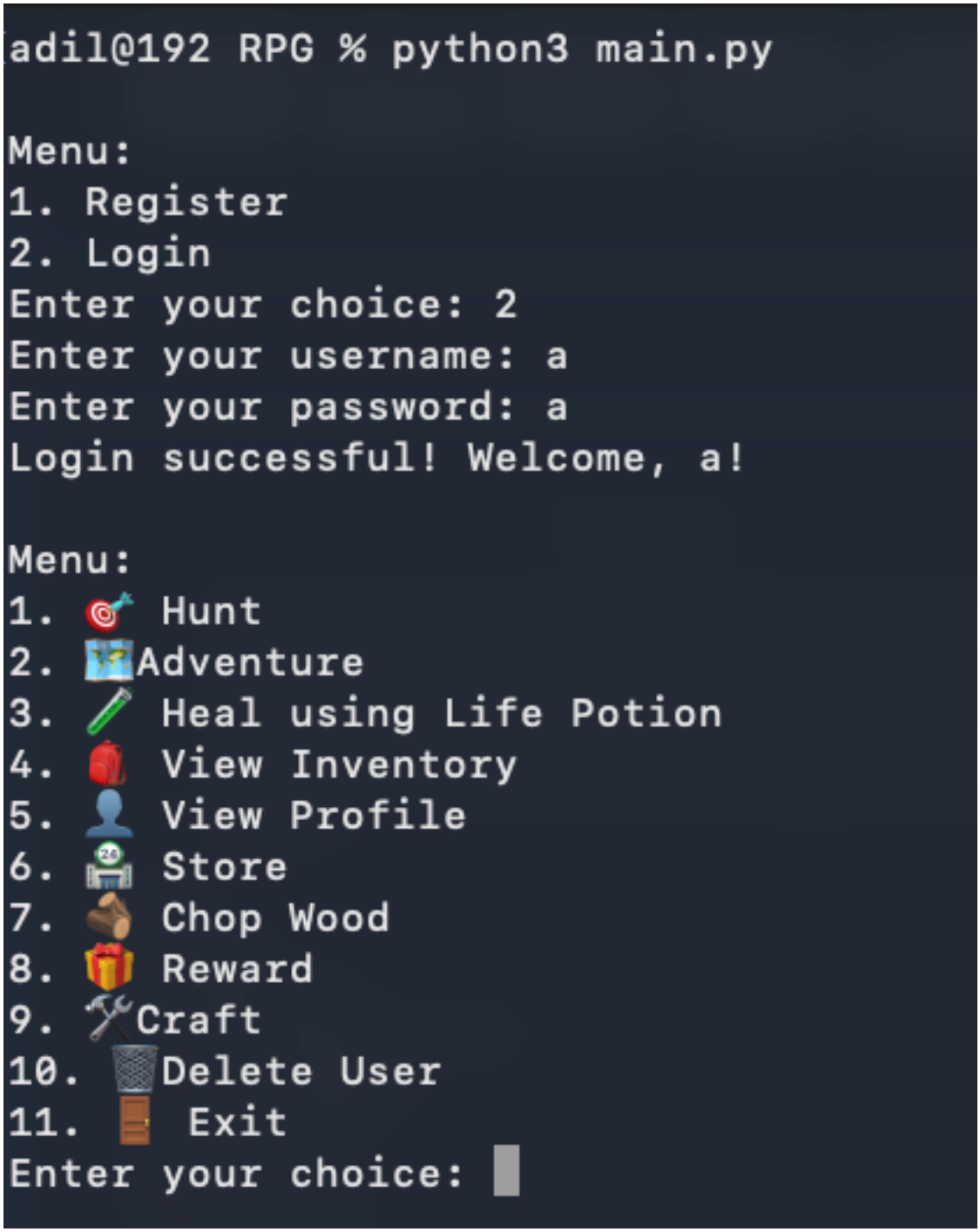

Figure 5.7 Login/Register/Main Menu

Hunt:

- During gameplay, the player hunts a deer enemy.The system updates health, XP, coins, and inventory in real time.Successful encounters contribute to player progression**.**

Figure 5.8 Hunt

Adventure:

- The player fights a Demonic Hydra, takes damage, and defeats the enemy. Rewards include XP, coins, and a level increase.

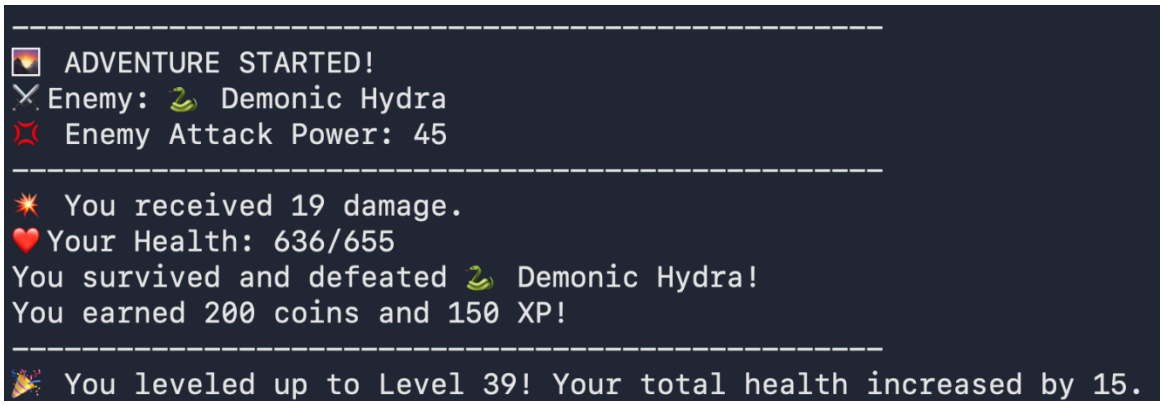

Figure 5.9 Adventure

Inventory:

- The inventory module maintains a list of all items obtained from battles and exploration.
  Each item is stored with quantity values and used for healing, crafting, or combat enhancement.

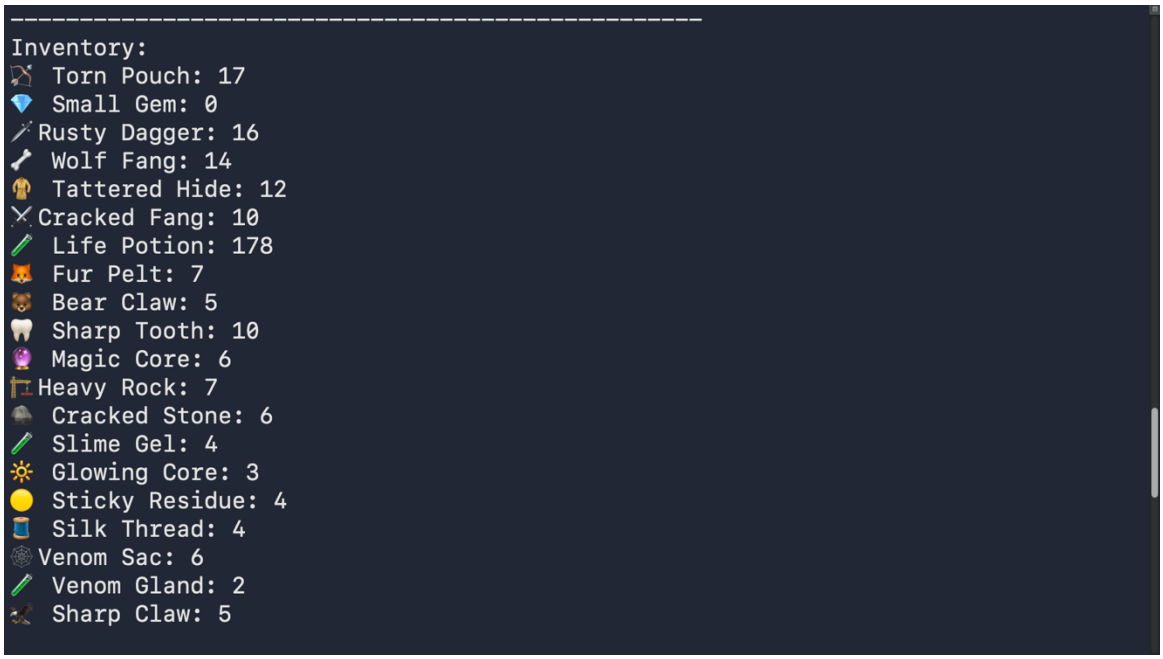

Figure 5.10 Inventory

Profile:

- The player profile module maintains the current character attributes, such as fighting stats, equipment, and resources. All the values are dynamically updated based on the actions performed during gameplay.

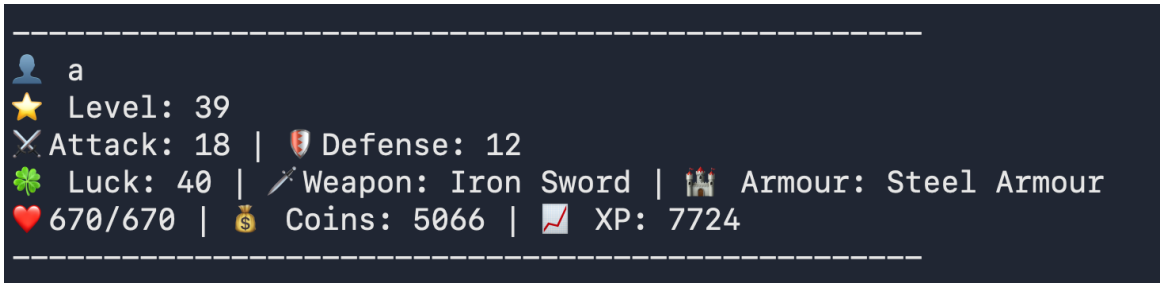

Figure 5.11 Profile

Store:

- The player has the capability to buy weapons, armor, and items (life potions) from the in-game store. The subtraction of coins is based on the price of the chosen item.

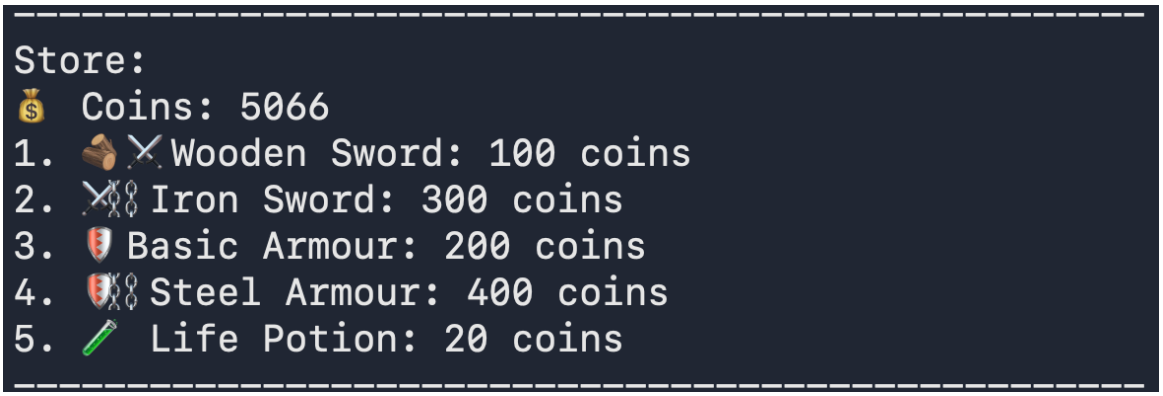

Figure 5.12 Store

Chop:

- The player gathers resources by interacting with the environment, and the gathered items are then added to the inventory.

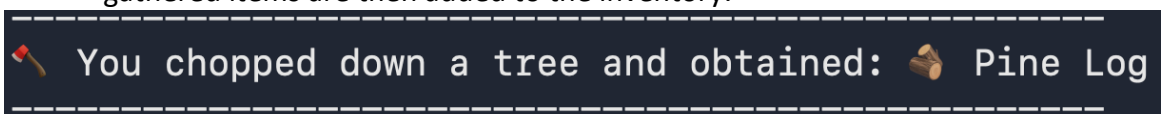

Figure 5.13 Chop

Reward:

- The reward system indicates how loot is distributed after an action is successfully executed.

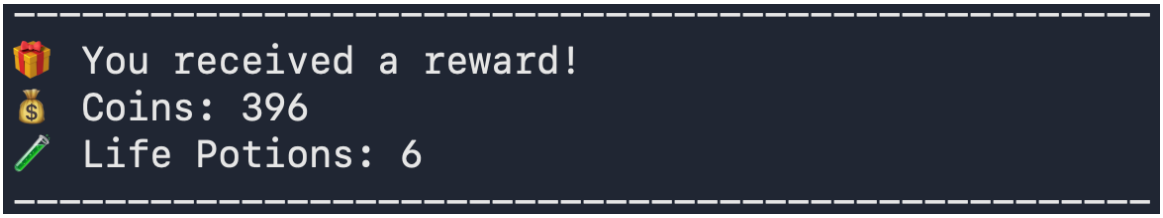

Figure 5.14 Reward

Craft:

- The player creates new items by using resources, and every item requires specific resources.

Figure 5.15 Craft

6. System Testing

6.1 Testing and Implementation

- The testing was carried out iteratively throughout the development process of the project by running the game on the terminal after implementing a new feature. The game was run after implementing every new feature to ensure that it was functioning properly.
- Whenever there were issues, such as crashes, incorrect results, or logical errors, the code was studied and corrected on the spot. This was a continuous process, ensuring that the game was functioning properly even while developing new features.

6.2 Testing Methodology

- The testing carried out was manual testing. The game was running on a command-line interface, and hence, testing was carried out by choosing different options on the menu and analyzing how the system reacted accordingly.
- Scenarios were considered while carrying out this evaluation, including how the game reacted normally, how it reacted with incorrect input, what happened if there were no items in the inventory, what happened if there was a cooldown, and what happened if there was a login failure.

6.3 Test Case Design

- The test cases were designed on the basis of the options available on the menu on the game screen. Every test case was designed to test a specific feature, and how the information related to the player, including health, experience points, items in the inventory, and statistics, was being updated was closely observed.

Test Case ID	Menu Option	Description	Expected Result
TC-01	Register	Create a new user account	User account created successfully
TC-02	Login	Login with valid credentials	User logged in
TC-03	Login	Login with invalid credentials	Access denied
TC-04	Hunt	Perform hunt action	XP, coins, and items updated
TC-05	Adventure	Perform adventure	Health reduced or rewards gained
TC-06	Chop Wood	Chop wood	Wood logs added to inventory
TC-07	Store	Buy weapon or item	Inventory updated, stats affected
TC-08	Craft	Craft Wooden Sword	Weapon crafted, attack increased
TC-09	Craft	Craft Lucky Charm	Luck increased, cooldown applied
TC-10	Heal	Use Life Potion	Health restored
TC-11	Inventory	View inventory	Items displayed correctly
TC-12	Profile	View profile	Player stats displayed
TC-13	Reward	Claim reward	XP or coins added
TC-14	Delete User	Delete account	User data removed
TC-15	Exit	Exit game	Program terminates safely

Table No. 6.1

6.4 System Implementation Strategy

- The system was developed in a step-by-step approach to ensure that the process of development remains simple. Basic functionalities such as user registration, login, and menu navigation were developed first.
- After the basic functionalities were developed, gameplay functionalities such as hunting, adventure, inventory management, and profile viewing were developed.

- Features such as the store, crafting system, cooldowns, and the luck attribute were added later. Each feature added to the system was tested using the terminal to see if it was working as expected.
- The process of implementing the features made it easier to detect bugs and correct them while ensuring the system was stable.

7. Conclusion

- This project helped in understanding how a role-playing game can be created at a basic level with the help of Python programming. While creating a text-based RPG, basic concepts in programming, like functions, data handling, file storage, etc., have been learned and implemented.

- The development of features like combat, inventory, crafting, etc., in this project was done in a way that it is simple to understand.

- This project has also helped in improving problem-solving skills, as there was a lot of error correction that took place during the development of this project.

- This project has thus been a success in terms of creating a simple RPG game with all the required features, along with providing a good learning experience in Python programming.

8. Learning During Project Work

- This project has been a learning experience for me, and I have learned how to divide a problem into smaller problems and how to implement it step by step.
- This project helped me in gaining practical knowledge in Python programming, especially in areas like file handling, functions, dictionaries, etc.
- This project has also helped me in improving my problem-solving skills, as there were a lot of errors that had to be corrected during the development of this project.
- This project has thus helped me in building confidence in programming, along with getting a better understanding of how real-world applications are develope

9. Online References

[1] W3Schools, “Python Tutorial,” W3Schools. [Online]. Available: https://www.w3schools.com/python/. Accessed throughout the project.

[2] EPIC RPG Wiki, “EPIC RPG Game Guide,” Fandom. [Online]. Available: https://epic-rpg.fandom.com/wiki/EPIC_RPG_Wiki. Accessed throughout the project.